

Midterm Exam Problem Statement

Scenario: Hospital Emergency Queue System

A small hospital is managing patients in the Emergency Room (ER). Because patient priority can change quickly (new critical patients arrive, discharged patients leave, etc.), the hospital wants a flexible system to keep track of the current patients in the ER.

You are asked to design and manipulate this ER queue using a **Doubly Linked List**.

Each node in the list represents **one patient**, and stores:

- patientID (integer)
- pointers to previous and next patients

You must support the following operations on this ER queue:

1. **Add a new patient at the beginning of the list**
 - This is used when a critical patient arrives by ambulance and must be treated first.
 - (Insert from beginning)
 2. **Add a new patient at the end of the list**
 - This is used for normal walk-in patients who can wait.
 - (Insert from ending)
 3. **Add a new patient at a specific position in the list**
 - The ER head nurse can insert a patient at an exact position k (1-based index), where:
 - position 1 means “treat immediately,”
 - position 2 means “treat after the first patient,” and so on.
 - For example, inserting at position 3 means: after the 2nd patient, before the 3rd patient.
 - (Insert at specific position)
 4. **Remove a patient from the beginning of the list**
 - This is used when the first patient has been treated and leaves the ER.
 - (Delete from beginning)
-

You're Tasks

Q1. (Implementation / Logic)

Write the logic (in C++ or in detailed pseudocode) for the following functions for this ER queue:

- `insertAtBeginning(patientID)`
- `insertAtEnd(patientID)`
- `insertAtPosition(patientID, position)`
- `deleteFromBeginning()`

Your code must clearly update:

- `head`
- `tail`
- `prev` and `next` pointers

You must handle edge cases, including:

- inserting into an empty list
- deleting when the list has only one node
- inserting at position 1
- inserting at a position greater than the current length (explain how you will handle it)

Q2. (Dry Run / Trace)

You start with an empty ER list.

Perform the following operations step by step, and after **each step**, draw the doubly linked list. Show each node's `patientID`, and show arrows for both `prev` and `next`.

Operations:

1. `insertAtEnd(101)`
2. `insertAtEnd(102)`
3. `insertAtBeginning(200)` // critical patient
4. `insertAtPosition(150, 2)`
5. `deleteFromBeginning()`
6. `insertAtEnd(300)`

After step 6, answer:

- (a) What is the `patientID` at the head?
- (b) What is the `patientID` at the tail?
- (c) Write the list from head → tail (forward traversal).
- (d) Write the list from tail → head (backward traversal).

Marks are given for correct order and correct pointer understanding.

Q3. Design the poster that has the following sections

1. Title
2. Sub Title (Student Name , Roll No)
3. Problem Statement
4. Proposed Solution
5. Graphical Representation of Linked List After Performing Each Step
 - `insertAtEnd(101)`
 - `insertAtEnd(102)`
 - `insertAtBeginning(200) // critical patient`
 - `insertAtPosition(150, 2)`
 - `deleteFromBeginning()`
 - `insertAtEnd(300)`

Upload the following on your GitHub directory

1. Complete C++ Code
2. Poster
3. Poster in Pdf as well